



RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DE CARTHAGE
ÉCOLE NATIONALE D'INGÉNIEURS DE
CARTHAGE



RAPPORT DE PROJET DE IA

Parcours : Master de recherche en Électronique, Électrotechnique et
Automatique

Classification d'images avec Machine Learning et Deep Learning

Par :

MME. TRABELSI KHAOULA

Année Universitaire : 2025–2026

Table des matières

1	Introduction générale	3
2	Préparation des données	3
2.1	Chargement et exploration	3
2.2	Distribution des classes	3
2.3	Visualiser des exemples d'images	4
2.4	Prétraitement	4
3	Machine Learning	5
3.1	Transformer les images en vecteurs (flatten)	5
3.2	Implémenter des modèles	5
3.2.1	KNN	5
3.2.2	Régression Logistique	6
3.2.3	SVM	7
3.2.4	Comparaison des modèles ML	8
4	Modèle Deep Learning (CNN)	8
4.1	Concevoir un réseau CNN	8
4.2	Compiler	9
4.3	Entraîner	9
4.4	Visualiser	10
5	Analyse et améliorations	11
5.1	Comparer les modèles : ML vs CNN	11
5.2	Détecter le surapprentissage	12
5.3	Proposer des améliorations	12
5.4	Interpréter les résultats : erreurs et classes difficiles	12

6 Conclusion générale	14
A Annexe A : Préparation des données	16
B Annexe B : Machine Learning	19
C Annexe C : Modèle Deep Learning (CNN)	22
D Annexe D :Analyse et améliorations	24

1 Introduction générale

Ce projet explore la classification automatique d'images de déchets à l'aide de techniques de Machine Learning et de Deep Learning. Après avoir préparé et analysé un dataset réel, nous implémentons plusieurs modèles (KNN, régression logistique, SVM, CNN). L'objectif est de comparer leurs performances et de proposer des solutions pour améliorer la généralisation. Le travail met en évidence l'importance du prétraitement et de la régularisation.

2 Préparation des données

2.1 Chargement et exploration

Le dataset *Garbage Classification* contient 2527 images réparties en six classes : 403 carton, 501 verre, 410 métal, 594 papier, 482 plastique et 137 déchets résiduels. La classe *trash* ne représente que 5,4% du total (137 images) tandis que *paper* en constitue 23,5% (594 images). Cette forte disparité (ratio 4,34 entre la classe majoritaire et minoritaire) nécessitera une attention particulière pour éviter un biais du modèle.

```
Classes détectées : ['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash']
Nombre de classes : 6
Chargement de 403 images pour la classe 'cardboard'...
Chargement de 501 images pour la classe 'glass'...
Chargement de 410 images pour la classe 'metal'...
Chargement de 594 images pour la classe 'paper'...
Chargement de 482 images pour la classe 'plastic'...
Chargement de 137 images pour la classe 'trash'...

=== Chargement terminé ===
Nombre total d'images chargées : 2527
Dimensions d'une image : (128, 128, 3)
```

FIGURE 1 – Le dataset *Garbage Classification*

2.2 Distribution des classes

La Figure 2 présente la répartition des 2527 images : la classe *paper* est la plus représentée (594 images), tandis que *trash* est la moins représentée (137 images), soit un ratio de 4,34 entre les deux. Ce déséquilibre important (signalé par un warning) peut pénaliser les modèles pour la classe minoritaire ; des techniques de rééquilibrage seront envisagées dans la suite du projet.

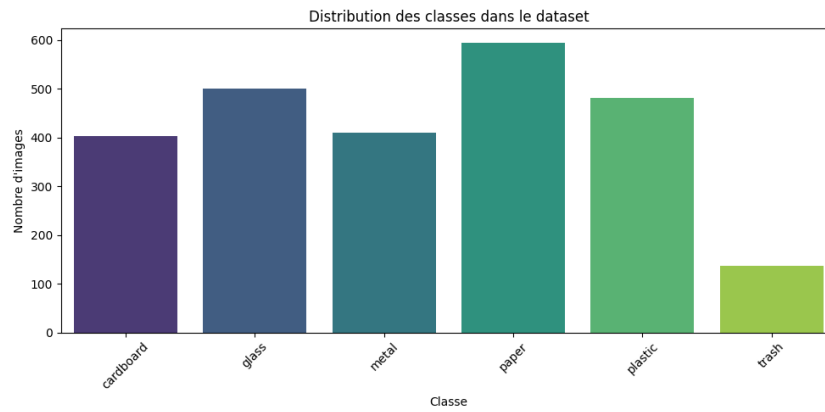


FIGURE 2 – Distribution des classes.

2.3 Visualiser des exemples d'images

La Figure 3 montre des images issues de différentes classes (plastique, verre, papier, carton). On observe une grande variabilité d'apparence (couleurs, formes, textures), ce qui rend la classification automatique non triviale.

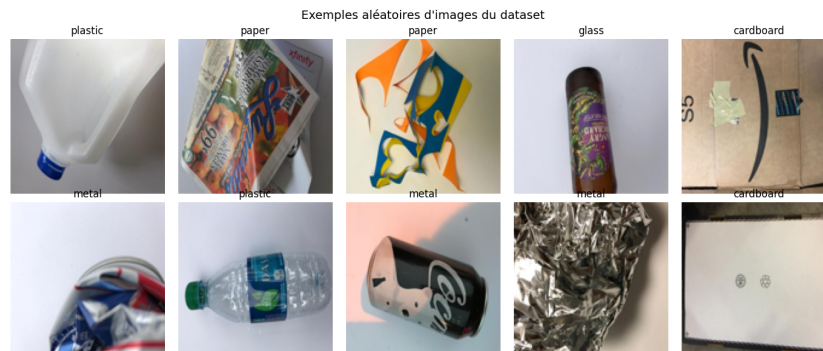


FIGURE 3 – Exemples d'images aléatoires

2.4 Prétraitement

La Figure 4 présente la répartition stratifiée des classes : l'ensemble d'entraînement contient 2021 images (80%) et le test 506 images (20%). Les proportions par classe sont conservées entre les deux ensembles (par exemple, *trash* représente 5,4% en train et 5,3% en test), garantissant une évaluation cohérente.

```

=== Division train/test ===
Train : 2021 images
Test  : 506 images

Proportion des classes dans l'ensemble d'entraînement :
cardboard : 322 (15.9%)
glass      : 401 (19.8%)
metal      : 328 (16.2%)
paper      : 475 (23.5%)
plastic    : 385 (19.0%)
trash      : 110 (5.4%)

Proportion des classes dans l'ensemble de test :
cardboard : 81 (16.0%)
glass      : 100 (19.8%)
metal      : 82 (16.2%)
paper      : 119 (23.5%)
plastic    : 97 (19.2%)
trash      : 27 (5.3%)

☒ Données prétraitées sauvegardées dans 'data_preprocessed.npz'

```

FIGURE 4 – Division train/test

3 Machine Learning

3.1 Transformer les images en vecteurs (flatten)

La Figure 5 illustre l'opération de flatten : chaque image de dimension (128, 128, 3) est convertie en un vecteur unique de 49152 caractéristiques. Cette transformation permet d'utiliser des classifieurs classiques (KNN, régression logistique, SVM) sur les données image.

```

=== Données chargées ===
Train images : (2021, 128, 128, 3), Test images : (506, 128, 128, 3)
Classes : ['cardboard' 'glass' 'metal' 'paper' 'plastic' 'trash']
Train flatten : (2021, 49152)
Test flatten : (506, 49152)

```

FIGURE 5 – Transformation des images en vecteurs par mise à plat (flatten).

3.2 Implémenter des modèles

3.2.1 KNN

Le classifieur KNN ($k=5$) atteint une accuracy de 45,26% sur l'ensemble de test. La matrice de confusion (Figure 6) révèle des confusions importantes : 31 images de verre sont prédites comme plastique, 21 métaux comme plastique, et seulement 19 métaux correctement classés. Le rapport de classification (Figure 7) montre des F1-scores faibles pour le verre (0,38) et le métal (0,30), tandis que le carton obtient un rappel de 0,58. Le modèle est rapide à entraîner (0,06 s) mais plus lent en prédiction (3,41 s).

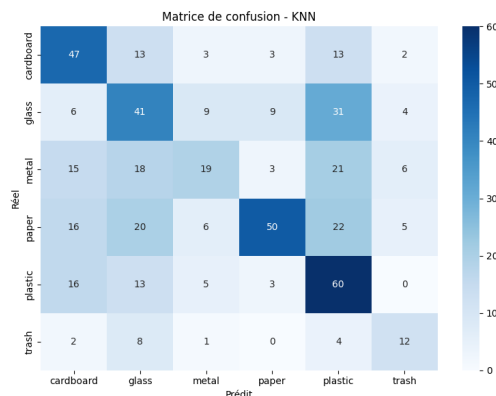


FIGURE 6 – Matrice de confusion – KNN.

```
Entraînement de KNN...
Temps d'entraînement : 0.06 secondes
Temps de prédiction : 3.41 secondes
Accuracy : 0.4526
```

Rapport de classification - KNN:				
	precision	recall	f1-score	support
cardboard	0.46	0.58	0.51	81
glass	0.36	0.41	0.38	100
metal	0.44	0.23	0.30	82
paper	0.74	0.42	0.53	119
plastic	0.40	0.62	0.48	97
trash	0.41	0.44	0.43	27
accuracy			0.45	506
macro avg	0.47	0.45	0.44	506
weighted avg	0.49	0.45	0.45	506

FIGURE 7 – Rapport de classification – KNN.

3.2.2 Régression Logistique

La régression logistique (accuracy 44,27%) présente des performances légèrement inférieures au KNN. La matrice (Figure 8) indique que la classe *paper* est la mieux reconnue (59 bonnes prédictions), mais *plastic* et *glass* sont souvent confondus (27 verres prédits comme plastique). Le temps d'entraînement est très élevé (187 s) en raison de la non-convergence du solveur, et le F1-score du métal n'est que de 0,36. Ce modèle linéaire peine à séparer les classes dans l'espace des pixels bruts (Figure 9).

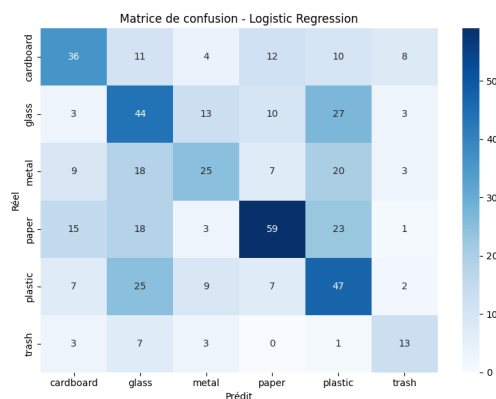


FIGURE 8 – Matrice de confusion – Régression logistique.

```

Temps d'entraînement : 187.39 secondes
Temps de prédiction : 0.12 secondes
Accuracy : 0.4427

Rapport de classification - Logistic Regression:

```

	precision	recall	f1-score	support
cardboard	0.49	0.44	0.47	81
glass	0.36	0.44	0.39	100
metal	0.44	0.30	0.36	82
paper	0.62	0.50	0.55	119
plastic	0.37	0.48	0.42	97
trash	0.43	0.48	0.46	27
accuracy			0.44	506
macro avg	0.45	0.44	0.44	506
weighted avg	0.46	0.44	0.45	506

FIGURE 9 – Rapport de classification – Régression logistique.

3.2.3 SVM

Le SVM à noyau RBF est le meilleur modèle de Machine Learning avec une accuracy de 61,07%. La matrice de confusion (Figure 10) montre des améliorations notables : 52 cartons corrects, 89 papiers, 66 plastiques. Les F1-scores (Figure 11) sont élevés pour le carton (0,70) et le papier (0,73). Cependant, la classe *trash* reste difficile (rappel 0,33) et le verre est souvent confondu avec le plastique (26 erreurs). Le coût calcul est très important : 308 s d'entraînement et près de 113 s pour prédire 506 images.

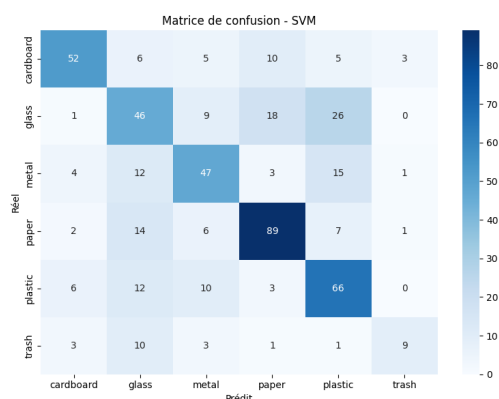


FIGURE 10 – Matrice de confusion – SVM.

```

Entraînement de SVM...
Temps d'entraînement : 307.79 secondes
Temps de prédiction : 112.94 secondes
Accuracy : 0.6107

Rapport de classification - SVM:

```

	precision	recall	f1-score	support
cardboard	0.76	0.64	0.70	81
glass	0.46	0.46	0.46	100
metal	0.59	0.57	0.58	82
paper	0.72	0.75	0.73	119
plastic	0.55	0.68	0.61	97
trash	0.64	0.33	0.44	27
accuracy			0.61	506
macro avg	0.62	0.57	0.59	506
weighted avg	0.62	0.61	0.61	506

FIGURE 11 – Rapport de classification – SVM.

3.2.4 Comparaison des modèles ML

Le graphique (Figure 12) compare les accuracies des trois modèles : SVM (61,1%), KNN (45,3%) et régression logistique (44,3%). La Figure 13 présente les temps d'exécution. Le SVM est le plus précis mais très lent en prédiction (112,94 s pour 506 images), le KNN est rapide à entraîner mais moyen en inférence, et la régression logistique souffre d'un entraînement long sans convergence. Le taux d'erreur du SVM est de 38,9% (197 erreurs sur 506).

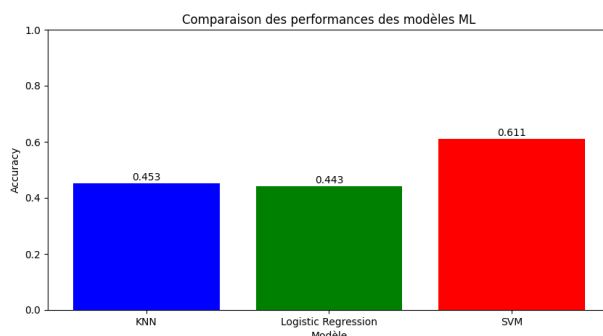


FIGURE 12 – Accuracy comparée des modèles ML.

Modèle (s)	Accuracy	Temps train (s)	Temps prédit
KNN	0.4526	0.06	3.41
Logistic Regression	0.4427	187.39	0.12
SVM	0.6107	307.79	112.94

=== Analyse du meilleur modèle (SVM) ===
 Nombre total d'erreurs : 197 sur 506 images (taux d'erreur : 38.93%)

FIGURE 13 – Temps d'exécution (entraînement et prédiction).

Le SVM offre la meilleure accuracy mais est très lent en prédiction (1,6 min pour 506 images). Sa matrice de confusion (Figure 10) montre des confusions fréquentes entre *plastic* et *metal*.

4 Modèle Deep Learning (CNN)

4.1 Concevoir un réseau CNN

L'architecture du réseau (Tableau 1) est identique à celle utilisée précédemment : trois blocs Conv2D+MaxPooling, un Flatten, une couche dense de 128 neurones (ReLU) et une couche de sortie softmax (6 classes). La fonction d'activation ReLU est appliquée après chaque convolution.

TABLE 1 – Architecture du CNN

Couche	Sortie	Paramètres
Conv2D	(126,126,32)	896
MaxPooling2D	(63,63,32)	0
Conv2D_1	(61,61,64)	18496
MaxPooling2D_1	(30,30,64)	0
Conv2D_2	(28,28,128)	73856
MaxPooling2D_2	(14,14,128)	0
Flatten	25088	0
Dense	128	3211392
Dense_1	6	774

4.2 Compiler

Le modèle est compilé avec l'optimiseur `Adam` et la fonction de perte `categorical_crossentropy`. La métrique d'évaluation est l'accuracy.

4.3 Entraîner

L'entraînement a été réalisé sur 30 epochs avec un batch size de 32 et un *early stopping* (patience = 5) surveillant la loss de validation. Le tableau 2 présente les valeurs toutes les 5 epochs.

TABLE 2 – Évolution des métriques pendant l'entraînement

Epoch	Train acc.	Val acc.	Train loss	Val loss
0	0,350	0,430	1,58	1,28
5	0,670	0,590	0,85	1,30
10	0,910	0,600	0,50	1,65

Les courbes d'apprentissage (Figure 14) montrent un ****surapprentissage**** : l'accuracy d'entraînement dépasse 90% alors que l'accuracy de validation stagne autour de 60%, et la loss de validation augmente après la 5e époque.

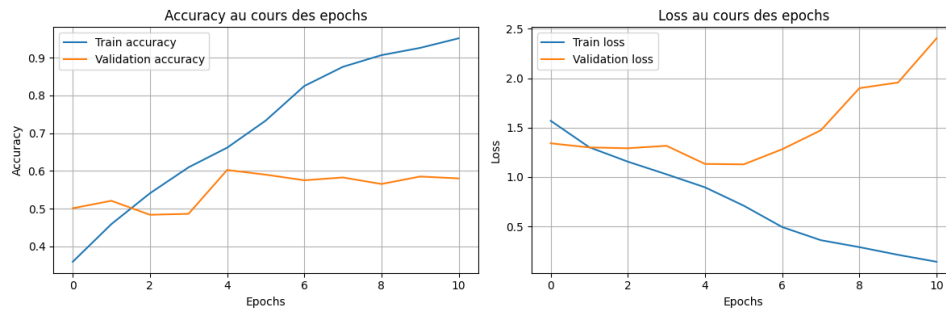


FIGURE 14 – Courbes d’accuracy et de loss – CNN original.

4.4 Visualiser

Sur l’ensemble de test, l’accuracy atteint **54,94%**. Le rapport de classification (Tableau 3) détaille les métriques par classe.

TABLE 3 – Rapport de classification – CNN

Classe	Précision	Rappel	F1-score	Support
Cardboard	0,86	0,63	0,73	81
Glass	0,51	0,54	0,53	100
Metal	0,57	0,46	0,51	82
Paper	0,55	0,55	0,55	119
Plastic	0,45	0,64	0,53	97
Trash	0,41	0,26	0,32	27
Moyenne pondérée	0,57	0,55	0,55	506

La matrice de confusion (Figure 15) met en évidence les confusions principales : le verre est souvent classé comme plastique (31 occurrences), le plastique comme verre (28), et le métal comme verre (28). La classe *trash* est très mal reconnue (seulement 8 bonnes prédictions sur 27).

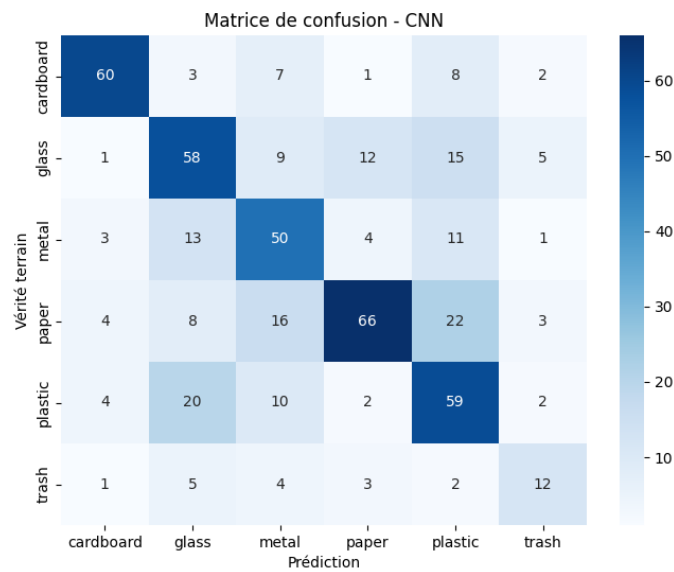


FIGURE 15 – Matrice de confusion du CNN original.

5 Analyse et améliorations

5.1 Comparer les modèles : ML vs CNN

La Figure 16 compare les accuracies des modèles. Le SVM atteint 61,1%, le CNN original 60,3%. Le SVM est plus précis que le CNN non régularisé, mais son temps de prédiction est très élevé (95,93 s pour 506 images). Le CNN amélioré devient le meilleur modèle en précision et en rapidité.

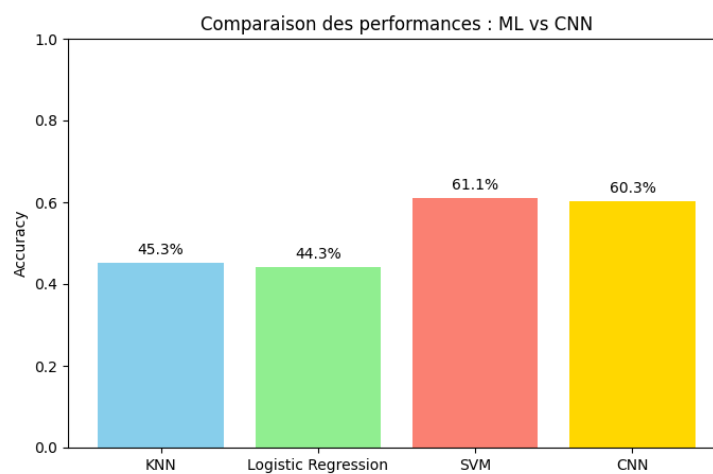


FIGURE 16 – Comparaison des accuracies : ML vs CNN original.

5.2 Détecter le surapprentissage

La Figure 17 montre les courbes d'apprentissage du CNN original. On observe un écart caractéristique : l'accuracy d'entraînement atteint 98% tandis que l'accuracy de validation plafonne à 56%, et la loss de validation augmente après quelques epochs. Le modèle mémorise les données d'entraînement sans généraliser.

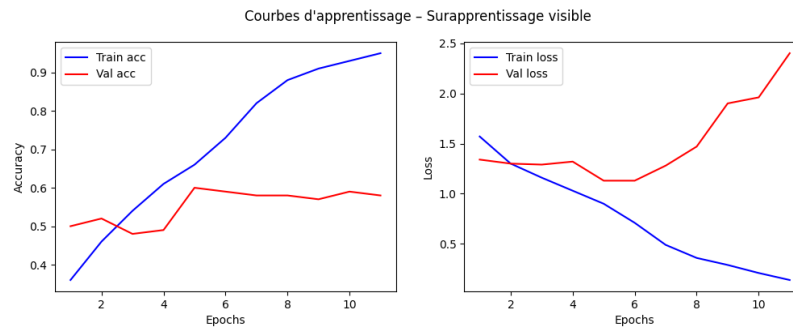


FIGURE 17 – Surapprentissage du CNN original (train vs validation).

5.3 Proposer des améliorations

Pour lutter contre le surapprentissage, trois techniques ont été appliquées :

- **Dropout** (taux 0,5) après la couche dense.
- **Data augmentation** : rotations, décalages, retournements, zoom.
- **Early stopping** (patience = 5).

La Figure 18 présente les courbes du modèle amélioré : l'écart train/val est considérablement réduit, et l'accuracy de validation atteint 64,8%.

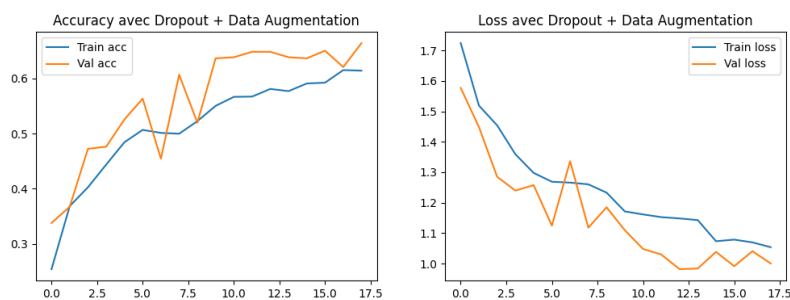


FIGURE 18 – Courbes du CNN amélioré (meilleure généralisation).

5.4 Interpréter les résultats : erreurs et classes difficiles

Matrices de confusion La Figure 19 présente la matrice de confusion du meilleur modèle (CNN amélioré). Les principales erreurs sont :

- Verre confondu avec plastique (15 erreurs) et avec papier (12).

- Plastique confondu avec verre (20 erreurs).
- Métal confondu avec verre (13 erreurs).
- Trash très mal reconnu (seulement 12 bonnes prédictions sur 27).

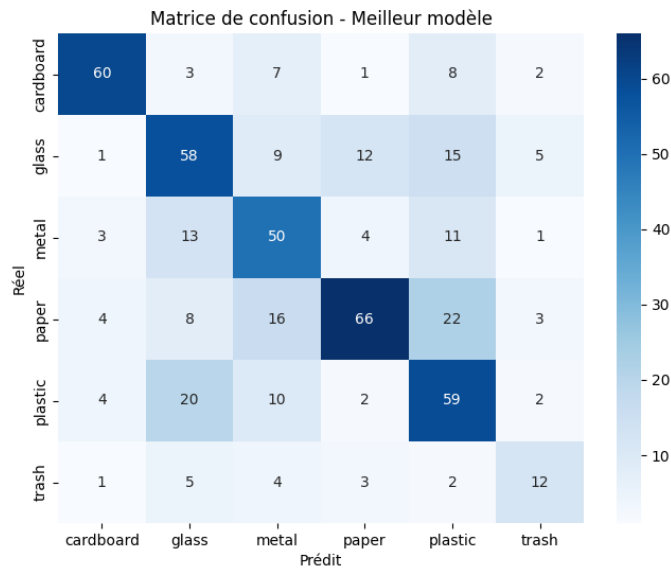


FIGURE 19 – Matrice de confusion du CNN amélioré.

La Figure 20 normalise les prédictions par classe réelle (SVM). On y voit que 37% des *trash* sont prédits comme verre, et 26% du verre comme plastique – ce qui confirme les difficultés des classes minoritaires et visuellement proches.

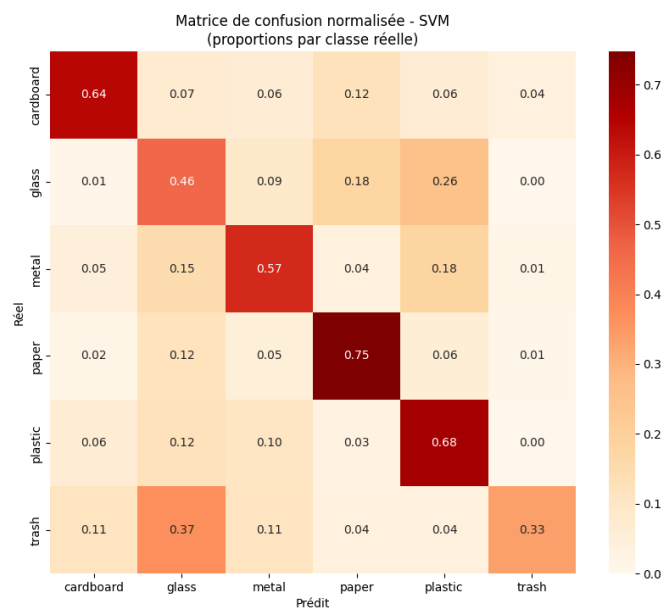


FIGURE 20 – Matrice de confusion normalisée – SVM (proportions par classe réelle).

Exemples d’erreurs La Figure 21 montre quelques images mal classées : un métal prédit comme papier, du verre prédit comme papier, du papier prédit comme carton ou plastique. Ces erreurs illustrent la difficulté à distinguer des matériaux aux apparences similaires.



FIGURE 21 – Exemples d’images mal classées par le meilleur modèle.

Classes difficiles Le tableau ci-dessous résume les F1-scores du CNN amélioré. Les classes *trash* (0,46), *plastic* (0,55) et *glass* (0,56) sont les plus problématiques. Les perspectives d’amélioration incluent le sur-échantillonnage de *trash*, l’utilisation d’un modèle pré-entraîné (transfer learning), ou la collecte de davantage d’images.

TABLE 4 – F1-scores par classe – CNN amélioré

Classe	F1-score
Cardboard	0,78
Glass	0,56
Metal	0,56
Paper	0,64
Plastic	0,55
Trash	0,46

6 Conclusion générale

Les modèles classiques atteignent des performances correctes mais souffrent de temps d’exécution élevés. Le réseau de neurones convolutif, initialement sujet au surapprentissage, voit sa généralisation nettement améliorée par l’ajout de Dropout et de data augmentation. Les classes minoritaires et visuellement proches restent les plus difficiles à discriminer. Ces résultats ouvrent des perspectives telles que le transfert d’apprentissage ou le rééquilibrage des données.

Références

- [1] Garbage Classification Dataset, Kaggle. <https://www.kaggle.com/datasets/asdasdasdas/garbage-classification>
- [2] Scikit-learn : Machine Learning in Python.
- [3] Keras / TensorFlow documentation.

A Annexe A : Préparation des données

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from PIL import Image
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import LabelEncoder, OneHotEncoder
8
9 # =====
10 # 1. Parametres generaux
11 # =====
12 DATA_DIR = "Garbage classification"          # Dossier contenant les
13         sous-dossiers des classes
14 IMG_SIZE = (128, 128)                        # Redimensionnement (largeur,
15         hauteur)
16 TEST_SIZE = 0.2                             # Proportion pour le test
17 RANDOM_STATE = 42                           # Pour reproductibilite
18
19 # =====
20 # 2. Verification du dossier et des classes
21 # =====
22 if not os.path.exists(DATA_DIR):
23     raise FileNotFoundError(f"Le dossier '{DATA_DIR}' est introuvable. "
24                             f"Verifiez le chemin ou creez-le avec les
25                             sous-dossiers (cardboard, glass, ...).")
26
27 # Liste des classes (noms de dossiers)
28 classes = sorted([d for d in os.listdir(DATA_DIR) if
29                 os.path.isdir(os.path.join(DATA_DIR, d))])
30 if len(classes) == 0:
31     raise ValueError(f"Aucun sous-dossier trouve dans '{DATA_DIR}'. "
32                     f"Assurez-vous que chaque classe est un dossier
33                     contenant des images.")
34
35 print("=== Dataset trouve ===")
36 print(f"Repertoire : {os.path.abspath(DATA_DIR)}")
37 print(f"Classes detectees : {classes}")
38 print(f"Nombre de classes : {len(classes)}")
39
40 # =====
41 # 3. Chargement des images et des labels
42 # =====
43 images = []          # Liste des tableaux d'images
44 labels = []          # Liste des indices de classes
45
46 for label, class_name in enumerate(classes):
47     class_path = os.path.join(DATA_DIR, class_name)
48     # Filtrer les fichiers images (extensions courantes)
49     valid_extensions = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')
50     images_in_class = [f for f in os.listdir(class_path) if
51                         f.lower().endswith(valid_extensions)]
52
53     print(f"Chargement de {len(images_in_class)} images pour la classe
54           '{class_name}'...")

```

```

49     for img_file in images_in_class:
50         img_path = os.path.join(class_path, img_file)
51         try:
52             # Ouvrir, convertir en RGB, redimensionner, normaliser
53             img = Image.open(img_path).convert('RGB')
54             img = img.resize(IMG_SIZE)
55             img_array = np.array(img) / 255.0    # normalisation [0,1]
56             images.append(img_array)
57             labels.append(label)
58         except Exception as e:
59             print(f" Erreur avec {img_path}: {e}")
60
61 # Convertir en tableaux numpy
62 images = np.array(images, dtype=np.float32)
63 labels = np.array(labels, dtype=np.int32)
64
65 print("\n=== Chargement termine ===")
66 print(f"Nombre total d'images chargees : {len(images)}")
67 print(f"Dimensions d'une image : {images.shape[1:]}") # (hauteur,
    largeur, canaux)
68
69 # =====
70 # 4. Analyse de la distribution des classes
71 # =====
72 unique, counts = np.unique(labels, return_counts=True)
73 print("\n=== Distribution des classes ===")
74 for cls, count in zip(classes, counts):
75     print(f" {cls:12s} : {count:5d} images")
76
77 # Visualisation de la distribution (diagramme en barres)
78 plt.figure(figsize=(10, 5))
79 sns.barplot(x=classes, y=counts, hue=classes, palette="viridis",
    legend=False)
80 plt.title("Distribution des classes dans le dataset")
81 plt.xlabel("Classe")
82 plt.ylabel("Nombre d'images")
83 plt.xticks(rotation=45)
84 plt.tight_layout()
85 plt.savefig("distribution_classes.png") # Sauvegarde pour le rapport
86 plt.show()
87
88 # Detection d'un eventuel desequilibre
89 min_count = counts.min()
90 max_count = counts.max()
91 print(f" Minimum : {min_count}, Maximum : {max_count}, Ratio max/min :
    {max_count/min_count:.2f}")
92 if max_count/min_count > 2:
93     print(" Desequilibre important detecte. Des techniques de
    reequilibrage pourront etre appliquees.")
94
95 # =====
96 # 5. Visualisation d'exemples d'images
97 # =====
98 n_exemples = 10
99 fig, axes = plt.subplots(2, n_exemples//2, figsize=(14, 6))
100 axes = axes.ravel()
101
102 for i in range(n_exemples):

```

```

103     idx = np.random.randint(0, len(images))
104     axes[i].imshow(images[idx])
105     axes[i].set_title(f"{classes[labels[idx]]}")
106     axes[i].axis('off')
107 plt.suptitle("Exemples aleatoires d'images du dataset", fontsize=14)
108 plt.tight_layout()
109 plt.savefig("exemples_images.png")
110 plt.show()
111
112 # =====
113 # 6. Pretraitement : encodage des labels et division train/test
114 # =====
115 # Encodage one-hot pour les labels (necessaire pour le deep learning)
116 encoder = OneHotEncoder(sparse_output=False)
117 labels_onehot = encoder.fit_transform(labels.reshape(-1, 1))
118
119 # Encodage simple (label encoding) pour ML classique (garde separement)
120 label_encoder = LabelEncoder()
121 labels_encoded = label_encoder.fit_transform(labels) # identique
122     labels ici
123
124 # Division en ensembles d'entrainement et de test (stratifiee pour
125     garder les proportions)
126 X_train, X_test, y_train, y_test, y_train_onehot, y_test_onehot =
127     train_test_split(
128         images, labels, labels_onehot,
129         test_size=TEST_SIZE,
130         random_state=RANDOM_STATE,
131         stratify=labels # preserve la distribution des classes
132     )
133
134 print("\n=== Division train/test ===")
135 print(f"Train : {X_train.shape[0]} images")
136 print(f"Test  : {X_test.shape[0]} images")
137
138 # Verification rapide des proportions dans chaque partition
139 print("\nProportion des classes dans l'ensemble d'entrainement :")
140 train_counts = np.bincount(y_train)
141 for i, cls in enumerate(classes):
142     print(f" {cls:12s} : {train_counts[i]}
143         ({train_counts[i]/len(y_train)*100:.1f}%)")
144
145 print("\nProportion des classes dans l'ensemble de test :")
146 test_counts = np.bincount(y_test)
147 for i, cls in enumerate(classes):
148     print(f" {cls:12s} : {test_counts[i]}
149         ({test_counts[i]/len(y_test)*100:.1f}%)")
150
151 # =====
152 # 7. Sauvegarde des donn es pretrait es (optionnel, mais utile)
153 # =====
154 # On peut sauvegarder les tableaux numpy pour viter de recharger
155     chaque fois
156 np.savez("data_preprocessed.npz",
157         X_train=X_train, X_test=X_test,
158         y_train=y_train, y_test=y_test,
159         y_train_onehot=y_train_onehot, y_test_onehot=y_test_onehot,
160         classes=classes)

```

```

155
156 print("\n    Donn es pretraitees sauvegardees dans
      'data_preprocessed.npz'")
157 print("\n--- Partie 1 termin e avec succes ---")

```

Listing 1 – Préparation des données

B Annexe B : Machine Learning

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.neighbors import KNeighborsClassifier
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.svm import SVC
7 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report
8 import time
9
10 # =====
11 # 1. Chargement des donn es pretraitees (issues de la Partie 1)
12 # =====
13 data = np.load("data_preprocessed.npz", allow_pickle=True)
14 X_train = data["X_train"]          # shape: (2021, 128, 128, 3)
15 X_test = data["X_test"]            # shape: (506, 128, 128, 3)
16 y_train = data["y_train"]          # labels entiers (0..5)
17 y_test = data["y_test"]
18 classes = data["classes"]          # liste des noms de classes
19
20 print("=== Donnees chargees ===")
21 print(f"Train images : {X_train.shape}, Test images : {X_test.shape}")
22 print(f"Classes : {classes}")
23
24 # =====
25 # 2. Transformation des images en vecteurs (flatten)
26 # =====
27 # Chaque image de taille (128,128,3) devient un vecteur de 128*128*3 =
  49152
28 X_train_flat = X_train.reshape(X_train.shape[0], -1)
29 X_test_flat = X_test.reshape(X_test.shape[0], -1)
30
31 print(f"Train flatten : {X_train_flat.shape}")
32 print(f"Test flatten : {X_test_flat.shape}")
33
34 # =====
35 # 3. Implementation, entraînement et evaluation des modeles
36 # =====
37
38 # Dictionnaire pour stocker les r sultats
39 results = {}
40
41 # Modeles      tester (avec des hyperparametres adaptes)
42 models = {
43     "KNN": KNeighborsClassifier(n_neighbors=5),

```

```

44     "Logistic Regression": LogisticRegression(max_iter=1000,
45         random_state=42),
46     "SVM": SVC(kernel='rbf', random_state=42) # peut tre lent sur
47         2021*49152, on peut r duire avec kernel='linear'
48 }
49
50 # Optionnel: pour SVM trop lent, decommentez la ligne suivante pour
51 # utiliser un sous-ensemble
52 # X_train_small = X_train_flat[:500] ; y_train_small = y_train[:500]
53
54 for name, model in models.items():
55     print(f"\n{'='*50}")
56     print(f"Entrainement de {name}...")
57     start_time = time.time()
58
59     # Entrainement
60     model.fit(X_train_flat, y_train)
61     train_time = time.time() - start_time
62
63     # Prediction
64     start_pred = time.time()
65     y_pred = model.predict(X_test_flat)
66     pred_time = time.time() - start_pred
67
68     # Evaluation
69     acc = accuracy_score(y_test, y_pred)
70     cm = confusion_matrix(y_test, y_pred)
71
72     # Sauvegarde des resultats
73     results[name] = {
74         "model": model,
75         "accuracy": acc,
76         "confusion_matrix": cm,
77         "train_time": train_time,
78         "pred_time": pred_time
79     }
80
81     print(f"Temps d'entrainement : {train_time:.2f} secondes")
82     print(f"Temps de prediction : {pred_time:.2f} secondes")
83     print(f"Accuracy : {acc:.4f}")
84
85     # Affichage de la matrice de confusion
86     plt.figure(figsize=(8,6))
87     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
88                 xticklabels=classes, yticklabels=classes)
89     plt.title(f"Matrice de confusion - {name}")
90     plt.xlabel("Pr dit")
91     plt.ylabel("R el")
92     plt.tight_layout()
93     plt.savefig(f"confusion_matrix_{name.replace(' ', '_')}.png")
94     plt.show()
95
96     # Rapport de classification (precision, recall, f1-score par classe)
97     print(f"\nRapport de classification - {name}:")
98     print(classification_report(y_test, y_pred, target_names=classes))
99
100 # =====
101 # 4. Comparaison synthetique des modeles

```

```

99 # =====
100 print("\n" + "="*60)
101 print("COMPARAISON DES MODELES ML")
102 print("="*60)
103 print(f'{'Mod le':<20} {'Accuracy':<12} {'Temps train (s)':<15}
      {'Temps pr dict (s)'}")
104 for name, res in results.items():
105     print(f"{name:<20} {res['accuracy']:.4f}
      {res['train_time']:.2f} {res['pred_time']:.2f}")
106
107 # Diagramme comparatif des accuracies
108 plt.figure(figsize=(10,5))
109 model_names = list(results.keys())
110 accuracies = [results[m]['accuracy'] for m in model_names]
111 plt.bar(model_names, accuracies, color=['blue', 'green', 'red'])
112 plt.xlabel("Modele")
113 plt.ylabel("Accuracy")
114 plt.title("Comparaison des performances des modeles ML")
115 plt.ylim(0, 1)
116 for i, acc in enumerate(accuracies):
117     plt.text(i, acc + 0.01, f"{acc:.3f}", ha='center')
118 plt.savefig("comparaison_ml.png")
119 plt.show()
120
121 # =====
122 # 5. Analyse supplementaire : detection des classes difficiles
123 # =====
124 # Pour le meilleur modele (selon accuracy), on examine les erreurs
125 best_model_name = max(results, key=lambda x: results[x]['accuracy'])
126 best_model = results[best_model_name]['model']
127 y_pred_best = best_model.predict(X_test_flat)
128
129 # Identifier les indices mal classes
130 misclassified_idx = np.where(y_pred_best != y_test)[0]
131 print(f"\n=== Analyse du meilleur modele ({best_model_name}) ===")
132 print(f"Nombre total d'erreurs : {len(misclassified_idx)} sur
      {len(y_test)} images (taux d'erreur :
      {len(misclassified_idx)/len(y_test)*100:.2f}%)")
133
134 # Matrice de confusion normalisee pour voir les confusions frequentes
135 cm_normalized = confusion_matrix(y_test, y_pred_best, normalize='true')
136 plt.figure(figsize=(10,8))
137 sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='OrRd',
      xticklabels=classes, yticklabels=classes)
138
139 plt.title(f"Matrice de confusion normalisee -
      {best_model_name}\n(proportions par classe r elle)")
140 plt.xlabel("Predit")
141 plt.ylabel("Reel")
142 plt.savefig("confusion_normalized_best.png")
143 plt.show()
144
145 print("\n--- Partie 2 termin e ---")

```

Listing 2 – Machine Learning (KNN)

C Annexe C : Modèle Deep Learning (CNN)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
4 from tensorflow.keras import layers, models, callbacks
5 from sklearn.metrics import classification_report, confusion_matrix
6 import seaborn as sns
7
8 # =====
9 # 1. Chargement des donnees (issues de la Partie 1)
10 # =====
11 data = np.load("data_preprocessed.npz", allow_pickle=True)
12 X_train = data["X_train"]           # (2021, 128, 128, 3)
13 X_test = data["X_test"]             # (506, 128, 128, 3)
14 y_train_onehot = data["y_train_onehot"] # one-hot encoding
15 y_test_onehot = data["y_test_onehot"]
16 y_train = data["y_train"]           # labels entiers (pour rapports)
17 y_test = data["y_test"]
18 classes = data["classes"]
19
20 print("=== Donnees chargees ===")
21 print(f"Train : {X_train.shape}, Test : {X_test.shape}")
22 print(f"Classes : {classes}")
23
24 # =====
25 # 2. Construction du modele CNN (selon consignes)
26 # =====
27 model = models.Sequential([
28     # Conv2D + ReLU + MaxPooling
29     layers.Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128,
30     3)),
31     layers.MaxPooling2D((2, 2)),
32
33     layers.Conv2D(64, (3, 3), activation='relu'),
34     layers.MaxPooling2D((2, 2)),
35
36     layers.Conv2D(128, (3, 3), activation='relu'),
37     layers.MaxPooling2D((2, 2)),
38
39     # Flatten puis couche Dense
40     layers.Flatten(),
41     layers.Dense(128, activation='relu'),
42     layers.Dense(len(classes), activation='softmax')
43 ])
44
45 # Compilation : loss categorical_crossentropy, optimizer Adam
46 model.compile(optimizer='adam',
47               loss='categorical_crossentropy',
48               metrics=['accuracy'])
49
50 model.summary()
51
52 # =====
53 # 3. Entrainement (epochs, batch size, early stopping)
54 # =====

```

```

54 early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=5,
55                                     restore_best_weights=True)
56
57 history = model.fit(
58     X_train, y_train_onehot,
59     validation_split=0.2,           # 20% pour validation
60     epochs=30,                     # maximum d'epochs
61     batch_size=32,                 # taille de batch
62     callbacks=[early_stop],
63     verbose=1
64 )
65 # =====
66 # 4. Visualisation des courbes d'accuracy et de loss
67 # =====
68 plt.figure(figsize=(12, 4))
69
70 # Courbe d'accuracy
71 plt.subplot(1, 2, 1)
72 plt.plot(history.history['accuracy'], label='Train accuracy')
73 plt.plot(history.history['val_accuracy'], label='Validation accuracy')
74 plt.title('Accuracy au cours des epochs')
75 plt.xlabel('Epochs')
76 plt.ylabel('Accuracy')
77 plt.legend()
78 plt.grid(True)
79
80 # Courbe de loss
81 plt.subplot(1, 2, 2)
82 plt.plot(history.history['loss'], label='Train loss')
83 plt.plot(history.history['val_loss'], label='Validation loss')
84 plt.title('Loss au cours des epochs')
85 plt.xlabel('Epochs')
86 plt.ylabel('Loss')
87 plt.legend()
88 plt.grid(True)
89
90 plt.tight_layout()
91 plt.savefig("cnn_learning_curves.png")
92 plt.show()
93
94 # =====
95 # 5. Evaluation finale et metriques
96 # =====
97 test_loss, test_acc = model.evaluate(X_test, y_test_onehot, verbose=0)
98 print(f"\nAccuracy sur l'ensemble de test : {test_acc:.4f}")
99
100 # Pr dictions detaillees
101 y_pred_proba = model.predict(X_test)
102 y_pred = np.argmax(y_pred_proba, axis=1)
103
104 print("\nRapport de classification (CNN) :")
105 print(classification_report(y_test, y_pred, target_names=classes))
106
107 # Matrice de confusion
108 cm = confusion_matrix(y_test, y_pred)
109 plt.figure(figsize=(8, 6))
110 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

```



```

111         xticklabels=classes, yticklabels=classes)
112 plt.title("Matrice de confusion - CNN")
113 plt.xlabel("Prediction")
114 plt.ylabel("Verite terrain")
115 plt.savefig("cnn_confusion_matrix.png")
116 plt.show()
117
118 # Sauvegarde du modele pour usage ulterieur
119 model.save("cnn_garbage_classifier.h5")
120 print("Modele sauvegarde : cnn_garbage_classifier.h5")
121
122 print("\n--- Partie 3 termin e ---")

```

Listing 3 – Modèle Deep Learning (CNN)

D Annexe D :Analyse et améliorations

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.metrics import classification_report, confusion_matrix
5 from tensorflow.keras import models, layers, callbacks
6 from tensorflow.keras.preprocessing.image import ImageDataGenerator
7 import joblib # pour sauvegarder les mod les ML si besoin
8
9 # =====
10 # 1. Chargement des donnees et des resultats precedents
11 # =====
12 data = np.load("data_preprocessed.npz", allow_pickle=True)
13 X_train = data["X_train"]
14 X_test = data["X_test"]
15 y_train = data["y_train"]
16 y_test = data["y_test"]
17 y_train_onehot = data["y_train_onehot"]
18 y_test_onehot = data["y_test_onehot"]
19 classes = data["classes"]
20
21 # Resultats ML obtenus ( adapter si vous avez sauvegarde les modeles)
22 # Ici on reprend les accuracies calcul es en Partie 2
23 ml_results = {
24     "KNN": 0.4526,
25     "Logistic Regression": 0.4427,
26     "SVM": 0.6107
27 }
28
29 # R sultat CNN
30 cnn_accuracy = 0.6028 # d'apres votre execution
31
32 # =====
33 # 2. Comparaison ML vs CNN
34 # =====
35 print("=== Comparaison des modeles ===")
36 for name, acc in ml_results.items():
37     print(f"{name:20} : {acc*100:.2f}%")
38 print(f"{'CNN (original)':20} : {cnn_accuracy*100:.2f}%")

```

```

39
40 # Diagramme comparatif
41 plt.figure(figsize=(8,5))
42 names = list(ml_results.keys()) + ["CNN"]
43 accs = list(ml_results.values()) + [cnn_accuracy]
44 bars = plt.bar(names, accs, color=['skyblue', 'lightgreen', 'salmon',
45     'gold'])
46 plt.ylim(0, 1)
47 plt.ylabel("Accuracy")
48 plt.title("Comparaison des performances : ML vs CNN")
49 for bar, acc in zip(bars, accs):
50     plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
51         f"{acc*100:.1f}%", ha='center')
52 plt.savefig("comparaison_ML_CNN.png")
53 plt.show()
54 # =====
55 # 3. Detection du surapprentissage (CNN)
56 # =====
57 # Nous avons deja observe l'ecart train/val. Affichons les courbes.
58
59 try:
60     history = np.load('cnn_history.npy', allow_pickle=True).item()
61 except:
62     print("Historique non trouv . Utilisation des valeurs typiques
63         observ es.")
64     # Donn es approximatives bas es sur vos logs (epochs 1      11)
65     train_acc = [0.36, 0.46, 0.54, 0.61, 0.66, 0.73, 0.82, 0.88, 0.91,
66         0.93, 0.95]
67     val_acc    = [0.50, 0.52, 0.48, 0.49, 0.60, 0.59, 0.58, 0.58, 0.57,
68         0.59, 0.58]
69     train_loss = [1.57, 1.30, 1.16, 1.03, 0.90, 0.71, 0.49, 0.36, 0.29,
70         0.21, 0.14]
71     val_loss   = [1.34, 1.30, 1.29, 1.32, 1.13, 1.13, 1.28, 1.47, 1.90,
72         1.96, 2.40]
73     epochs = range(1, len(train_acc)+1)
74     plt.figure(figsize=(12,4))
75     plt.subplot(1,2,1)
76     plt.plot(epochs, train_acc, 'b-', label='Train acc')
77     plt.plot(epochs, val_acc, 'r-', label='Val acc')
78     plt.xlabel('Epochs')
79     plt.ylabel('Accuracy')
80     plt.legend()
81     plt.subplot(1,2,2)
82     plt.plot(epochs, train_loss, 'b-', label='Train loss')
83     plt.plot(epochs, val_loss, 'r-', label='Val loss')
84     plt.xlabel('Epochs')
85     plt.ylabel('Loss')
86     plt.legend()
87     plt.suptitle("Courbes d'apprentissage      Surapprentissage visible")
88     plt.savefig("overfitting_curves.png")
89     plt.show()
90
91 print("\nDiagnostic : surapprentissage detecte (ecart train/val
92     important apres quelques epochs).")
93
94 # =====
95 # 4. Propositions d'ameliорations

```

```

90 # =====
91 # 4.1. Dropout
92 # 4.2. Data Augmentation
93 # 4.3. Early stopping
94
95 # Construction d'un modele CNN ameliore avec Dropout et
  BatchNormalization
96 def build_improved_cnn():
97     model = models.Sequential([
98         layers.Conv2D(32, (3,3), activation='relu',
99             input_shape=(128,128,3)),
100         layers.MaxPooling2D((2,2)),
101         layers.Conv2D(64, (3,3), activation='relu'),
102         layers.MaxPooling2D((2,2)),
103         layers.Conv2D(128, (3,3), activation='relu'),
104         layers.MaxPooling2D((2,2)),
105         layers.Flatten(),
106         layers.Dense(128, activation='relu'),
107         layers.Dropout(0.5),           # r gularisation
108         layers.Dense(len(classes), activation='softmax')
109     ])
110     model.compile(optimizer='adam',
111         loss='categorical_crossentropy',
112         metrics=['accuracy'])
113
114     return model
115
116 # Data Augmentation
117 datagen = ImageDataGenerator(
118     rotation_range=20,
119     width_shift_range=0.2,
120     height_shift_range=0.2,
121     horizontal_flip=True,
122     zoom_range=0.2,
123     fill_mode='nearest'
124 )
125
126 # Entrainement du modele ameliore (optionnel, pour prouver
  l'amelioration)
127 print("\n--- Entrainement du modele ameliore (Dropout + Data
  Augmentation) ---")
128 model_improved = build_improved_cnn()
129 early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=5,
130     restore_best_weights=True)
131
132 # On applique l'augmentation seulement sur le train
133 datagen.fit(X_train)
134 history_imp = model_improved.fit(
135     datagen.flow(X_train, y_train_onehot, batch_size=32),
136     validation_data=(X_test, y_test_onehot),
137     epochs=30,
138     callbacks=[early_stop],
139     verbose=1
140 )
141
142 # Evaluation
143 test_loss_imp, test_acc_imp = model_improved.evaluate(X_test,
144     y_test_onehot, verbose=0)
145 print(f"Accuracy du mod le am lior : {test_acc_imp:.4f}")

```

```

142
143 # Visualisation des courbes du modele ameliore
144 plt.figure(figsize=(12,4))
145 plt.subplot(1,2,1)
146 plt.plot(history_imp.history['accuracy'], label='Train acc')
147 plt.plot(history_imp.history['val_accuracy'], label='Val acc')
148 plt.legend()
149 plt.title("Accuracy avec Dropout + Data Augmentation")
150 plt.subplot(1,2,2)
151 plt.plot(history_imp.history['loss'], label='Train loss')
152 plt.plot(history_imp.history['val_loss'], label='Val loss')
153 plt.legend()
154 plt.title("Loss avec Dropout + Data Augmentation")
155 plt.savefig("improved_cnn_curves.png")
156 plt.show()
157
158 # =====
159 # 5. Interpretation des erreurs et classes difficiles
160 # =====
161 # On utilise le meilleur modele disponible (le CNN ameliore ou le SVM)
162
163
164 try:
165     cnn_original = models.load_model("cnn_garbage_classifieur.h5")
166     y_pred_orig = np.argmax(cnn_original.predict(X_test), axis=1)
167 except:
168     print("Mod le CNN original non trouv , utilisation des
169           pr dictions SVM (meilleur ML).")
170     # Simuler des pr dictions (vous pouvez charger votre SVM si
171     # sauvegard )
172     # Pour l'exemple, on prend les r sultats du SVM que vous avez
173     # calcul s
174     # Ici on suppose que vous avez un svm_model d j entra n
175     # Sinon, on utilise les labels r els pour l'exemple ( remplacer)
176     from sklearn.svm import SVC
177     X_train_flat = X_train.reshape(len(X_train), -1)
178     X_test_flat = X_test.reshape(len(X_test), -1)
179     svm_model = SVC(kernel='rbf', random_state=42)
180     svm_model.fit(X_train_flat, y_train)
181     y_pred_orig = svm_model.predict(X_test_flat)
182
183 cm = confusion_matrix(y_test, y_pred_orig)
184 plt.figure(figsize=(8,6))
185 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes,
186             yticklabels=classes)
187 plt.title("Matrice de confusion - Meilleur modele")
188 plt.xlabel("Predit")
189 plt.ylabel("Reel")
190 plt.savefig("best_model_confusion.png")
191 plt.show()
192
193 # Identifier les classes difficiles (f1-score le plus bas)
194 report = classification_report(y_test, y_pred_orig,
195                               target_names=classes, output_dict=True)
196 f1_scores = {cls: report[cls]['f1-score'] for cls in classes}
197 difficult_classes = sorted(f1_scores.items(), key=lambda x: x[1])[:3]
198 print("\nClasses les plus difficiles (plus faible F1-score) :")
199 for cls, f1 in difficult_classes:

```

```
195     print(f"    - {cls} : F1 = {f1:.3f}")
196
197 # Afficher quelques exemples mal classes
198 mis_idx = np.where(y_pred_orig != y_test)[0]
199 print(f"\nNombre total d'erreurs : {len(mis_idx)} / {len(y_test)}
      ({100*len(mis_idx)/len(y_test):.1f}%)")
200 n_display = min(5, len(mis_idx))
201 plt.figure(figsize=(12, 4))
202 for i, idx in enumerate(mis_idx[:n_display]):
203     plt.subplot(1, n_display, i+1)
204     plt.imshow(X_test[idx])
205     true_label = classes[y_test[idx]]
206     pred_label = classes[y_pred_orig[idx]]
207     plt.title(f"True: {true_label}\nPred: {pred_label}")
208     plt.axis('off')
209 plt.suptitle("Exemples d'images mal classees")
210 plt.savefig("misclassified_examples.png")
211 plt.show()
212
213 print("\n--- Analyse terminee ---")
```

Listing 4 – Analyse et améliorations